

NewsGenie

An AI-Powered Information and News Assistant

Course End Project Report

Course	Agentic AI / Applied AI Systems
Project	NewsGenie – AI-Powered News Assistant
Stack	Python · Streamlit · LangGraph · OpenAI GPT-4o-mini · NewsAPI · Tavily
Date	May 2026

Table of Contents

EXECUTIVE SUMMARY	4
1. INTRODUCTION	5
1.1 PROBLEM STATEMENT	5
1.2 OBJECTIVES	5
1.3 SCOPE	5
2. SYSTEM ARCHITECTURE	6
2.1 THREE-LAYER ARCHITECTURE	6
2.2 TECHNOLOGY STACK	6
3. MODULE DESIGN	7
3.1 STATE.PY	7
3.2 AGENTS.PY	7
3.3 WORKFLOW.PY	7
3.4 MAIN.PY	7
4. LANGGRAPH QUERY ROUTING WORKFLOW	8
4.1 GRAPH STRUCTURE	8
4.2 NODE DESCRIPTIONS	9
Node 1 — <i>classify_query</i>	9
Node 2 — <i>fetch_news</i>	9
Node 3 — <i>llm_answer</i>	9
Node 4 — <i>score_news</i>	9
Node 5 — <i>format_response</i>	9
5. CHATBOT DESIGN AND CONVERSATION MANAGEMENT	10
5.1 QUERY CLASSIFICATION	10
5.2 CONTEXT-AWARE CLASSIFICATION	10
5.3 CONVERSATION HISTORY	10
5.4 SESSION CONTINUITY	10
6. API AND WEB SEARCH INTEGRATION	11
6.1 NEWSAPI INTEGRATION	11
6.2 LANDING PAGE DIRECT CALLS	11
6.3 TAVILY WEB SEARCH	11
6.4 CREDENTIAL MANAGEMENT	11
7. LLM-POWERED CREDIBILITY FILTERING	12
7.1 DESIGN DECISION — WHY LLM OVER A SEPARATE ML MODEL	12
7.2 CREDIBILITY ASSESSMENT SIGNALS	12
7.3 SCORING AND FILTERING LOGIC	13
7.4 WHY FETCH 10, SHOW UP TO 5	13
7.5 CONSISTENCY ACROSS ALL SURFACES	13
7.6 IMPORTANT LIMITATIONS	13
8. SESSION MEMORY AND STATE MANAGEMENT	14
8.1 STREAMLIT SESSION STATE	14
8.2 LANGGRAPH STATE VS SESSION STATE	14
9. ERROR HANDLING AND FALLBACK CHAIN	15

9.1 THREE-LEVEL FALLBACK	15
9.2 ERROR SCENARIOS	15
10. USER INTERFACE DESIGN	16
10.1 WELCOME SCREEN	16
10.2 CHAT VIEW	16
10.3 SIDEBAR	16
10.4 DOUBLE-RENDER BUG FIX	16
11. EXECUTION FLOW — END-TO-END TRACE	17
12. SAMPLE OUTPUTS	18
12.1 NEWS QUERY — TOPIC PILL CLICK	18
12.2 NEWS QUERY — FREE TEXT	18
12.3 GENERAL QUERY	18
12.4 FALLBACK SCENARIO	18
13. LIMITATIONS AND FUTURE ENHANCEMENTS	19
13.1 CURRENT LIMITATIONS	19
13.2 FUTURE ENHANCEMENTS	19
14. CONCLUSION	20
APPENDIX A — DEPENDENCIES AND SETUP	21
A.1 REQUIREMENTS.TXT	21
A.2 API KEYS REQUIRED	21
A.3 RUN INSTRUCTIONS	21
A.4 FILE STRUCTURE	21

Executive Summary

NewsGenie is an AI-powered conversational news assistant that addresses three problems in modern digital information consumption: fragmented news sources, the proliferation of misinformation, and the need to answer both real-time news queries and general knowledge questions from a single interface.

The system is built on four pillars. First, real-time news retrieval via NewsAPI.org, covering Business, Technology, Sports, Health, and Entertainment categories. Second, intelligent query routing via a LangGraph StateGraph that classifies every user query as either a news request or a general knowledge question and routes it down the appropriate processing path. Third, LLM-powered credibility filtering: every batch of fetched articles is assessed by GPT-4o-mini against four credibility signals — language tone, source attribution, claim specificity, and headline integrity. Articles scored UNRELIABLE are dropped before the user sees them; survivors are ranked with RELIABLE articles first. Fourth, a Streamlit conversational interface with session-aware chat history, topic-based navigation, and transparent credibility labelling on every article card.

The system implements a three-level fallback chain — NewsAPI, then Tavily web search, then LLM-only response — ensuring continuous service even when external APIs fail. All credibility filtering is performed by the same LLM already used for classification and answering, eliminating the need for a separate ML model, additional dependencies, or local GPU infrastructure.

1. Introduction

1.1 Problem Statement

Modern news consumption is fragmented, overwhelming, and increasingly unreliable. Users must navigate multiple platforms, filter misinformation manually, and switch between different tools to answer both current-events questions and general knowledge queries. No single conversational tool solves all three problems simultaneously.

"People struggle to keep up with real-time, reliable news in today's fast-paced world. Users face challenges such as filtering misinformation, finding trustworthy sources, and securing timely answers to general queries — all in one place."
— Course Project Brief

1.2 Objectives

1. Build an AI chatbot that classifies and routes queries between live news retrieval and LLM knowledge responses.
2. Integrate NewsAPI.org for real-time news and Tavily for web search fallback.
3. Implement a LangGraph StateGraph to orchestrate query processing with conditional routing.
4. Filter misinformation by scoring all fetched articles for credibility before display, using GPT-4o-mini as the assessor.
5. Deploy a Streamlit frontend with session-aware conversation, topic navigation, and transparent credibility labelling.
6. Implement robust error handling with a three-level fallback chain.

1.3 Scope

The system processes English-language queries and news articles. It does not implement user authentication or cross-session persistence. Credibility scoring uses article title and description only, as the NewsAPI free tier does not provide full article body text. These limitations are documented in Section 13.

2. System Architecture

2.1 Three-Layer Architecture

NewsGenie is structured as three cleanly separated layers. The UI layer handles all Streamlit rendering and session state. The Brain layer orchestrates all query processing through LangGraph nodes. The API layer provides external data.

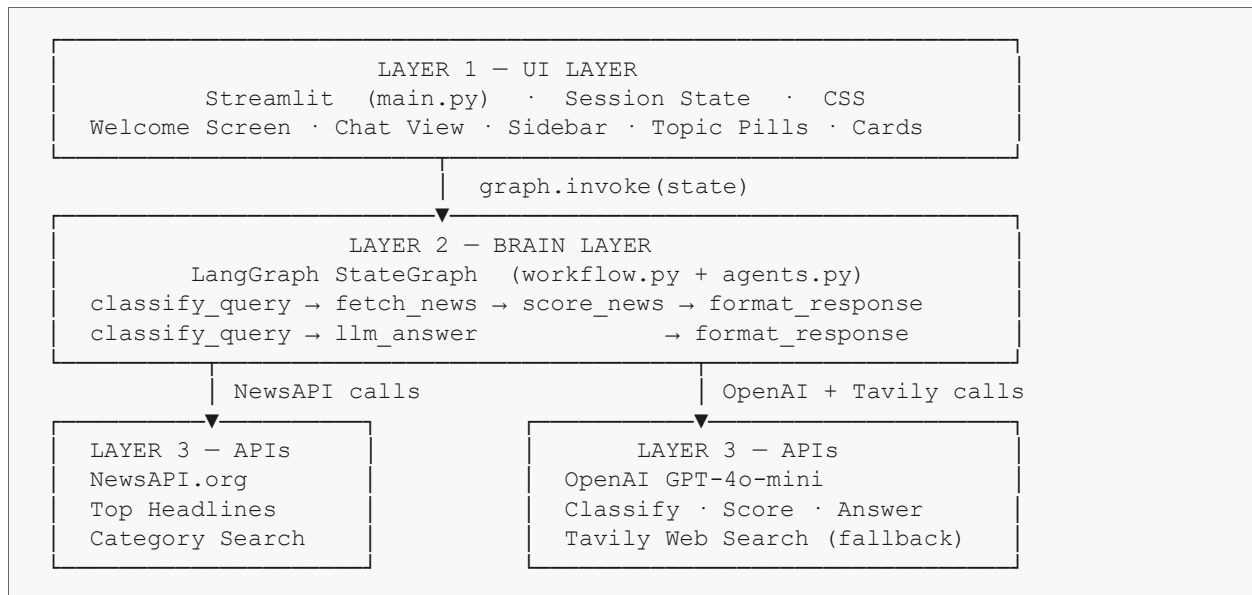


Figure 2.1 — NewsGenie Three-Layer Architecture

2.2 Technology Stack

Component	Technology	Role
Frontend	Streamlit ≥ 1.32	Reactive chat UI, session state, layout
Orchestration	LangGraph ≥ 0.1	StateGraph, conditional routing, node execution
LLM	OpenAI GPT-4o-mini	Classification, credibility scoring, general answers, fallback summaries
LangChain	langchain-openai	ChatOpenAI wrapper, SystemMessage / HumanMessage
News API	NewsAPI.org	Real-time top headlines and category news (10 articles per fetch)
Web Search	Tavily Python SDK	News fallback + general query enrichment (RAG context)
Env Management	python-dotenv	Secure API key loading from .env file

Table 2.1 — Technology Stack

3. Module Design

The codebase is divided into four Python modules, each with a single clearly defined responsibility. There are no circular imports. Each module can be read, tested, and reasoned about independently.

File	Size	Responsibility
state.py	~75 lines	NewsGenieState TypedDict, NewsArticle shape, empty_state() factory
agents.py	~500 lines	All 5 LangGraph node functions, score_and_filter(), 3 landing page helpers
workflow.py	~45 lines	StateGraph wiring, conditional routing, build_graph()
main.py	~460 lines	Streamlit UI: welcome screen, chat view, sidebar, CSS, session management

Table 3.1 — Module Summary

3.1 state.py

Defines the shared data contract. NewsGenieState is a TypedDict with 17 fields spanning input, classification result, fetch results, credibility filter outcome, LLM answer, and formatted response. NewsArticle carries two credibility fields — credibility_label (RELIABLE / UNCERTAIN / UNRELIABLE) and credibility_reason (one-sentence LLM explanation) — both None until score_news runs. The empty_state() factory creates a clean state for each new query.

3.2 agents.py

Contains all five LangGraph node functions (classify_query, fetch_news, score_news, llm_answer, format_response), the shared score_and_filter() function used by both the graph and the landing page helpers, and three landing page helper functions that bypass the graph and call the NewsAPI and scorer directly. The _get_llm() factory is the only place that constructs a ChatOpenAI instance, ensuring consistent model configuration across all nodes.

3.3 workflow.py

Imports the five node functions and wires them into a LangGraph StateGraph. Defines route_after_classify() — the conditional routing function — and exposes a single public function build_graph(). The compiled graph is cached in st.session_state at startup to avoid recompilation on every Streamlit rerun.

3.4 main.py

The only file that imports Streamlit. Manages session state initialisation, renders the sidebar and either the welcome screen or chat view depending on in_chat, and calls run_query() for every user interaction requiring a graph response. The _apply_landing_fetch() function handles topic pill and quick action button clicks with a st.rerun() at the end to prevent mid-render state mutation — the fix for the double-render bug discovered during testing.

4. LangGraph Query Routing Workflow

4.1 Graph Structure

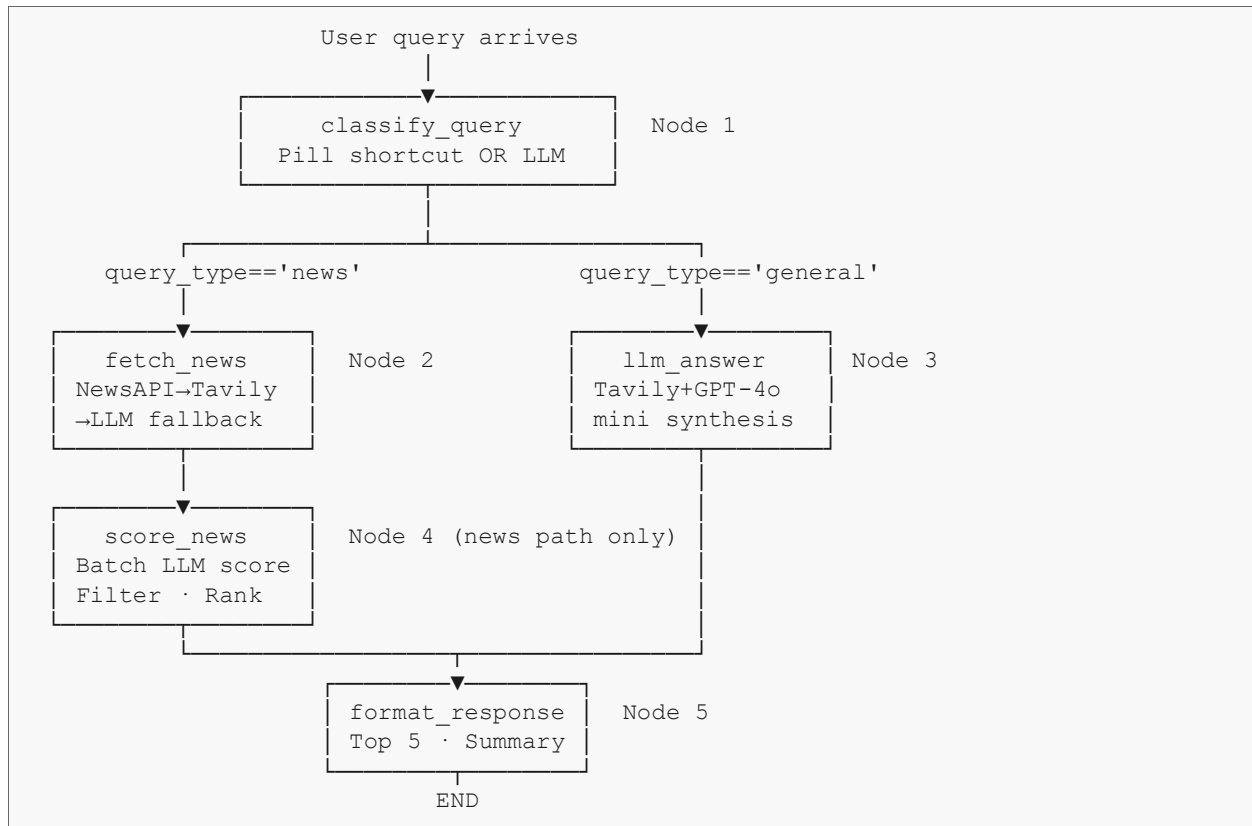


Figure 4.1 — LangGraph StateGraph

4.2 Node Descriptions

Node 1 — `classify_query`

Determines whether the query requires live news or can be answered from LLM knowledge. Two paths: if `selected_category` is set (user clicked a topic pill), immediately returns `query_type='news'` and `extracted_topics=[category]` with no LLM call — saving one API call and 400-800ms latency. For free-text queries, calls GPT-4o-mini with a structured classification prompt that includes the last four messages of chat history as context, enabling correct classification of follow-up queries such as 'tell me more about that' or 'explain the first story.'

Node 2 — `fetch_news`

Fetches up to 10 articles via a three-level fallback chain. Always fetches 10 (not 5) to give the credibility filter room to drop UNRELIABLE articles while still leaving enough to show the user five results. Detailed in Section 9.

Node 3 — `llm_answer`

Handles general queries using a RAG pattern. Calls Tavily with `search_depth='advanced'` to retrieve up to five web snippets, injects them as context into the GPT-4o-mini prompt along with the last six turns of chat history, and synthesises a grounded answer. Runs only on the general path — `score_news` is skipped entirely for this branch since there are no articles to score.

Node 4 — `score_news`

The credibility filtering node. Runs exclusively on the news path, between `fetch_news` and `format_response`. Calls `score_and_filter()` which sends all fetched articles to GPT-4o-mini in a single batch call. Articles scored UNRELIABLE are dropped. Survivors are ranked RELIABLE first, UNCERTAIN second, with recency preserved within each group. Records the count of dropped articles in `filtered_count` so `format_response` can notify the user. Skipped when `used_llm_fallback` is True.

Node 5 — `format_response`

Assembles the final output consumed by Streamlit. For the news path, articles are already filtered and ranked by `score_news` — `format_response` simply takes `articles[:5]`, deduplicates source names, and builds the summary string including the filter count note. For the general path, passes through `llm_response` directly. For the fallback path, calls the LLM to generate a knowledge-based summary with a disclaimer.

5. Chatbot Design and Conversation Management

5.1 Query Classification

The classifier uses a minimal, example-driven prompt that instructs GPT-4o-mini to return a single JSON object with type and topics fields. Four concrete examples are provided in the prompt rather than abstract rules — this consistently improves JSON output quality. The prompt is kept short and ends with 'No markdown. JSON only.' Post-processing strips any residual markdown fences before JSON parsing. On parse failure, the system defaults to general so the user always receives a response.

5.2 Context-Aware Classification

The last four messages of chat history (two exchanges) are prepended to the classifier's user message. This enables correct classification of follow-up queries. Without context, 'tell me more about that' would be classified as general (no news keywords). With context, the classifier sees the prior assistant response was about a news topic and correctly routes the follow-up to `fetch_news`.

5.3 Conversation History

Two parallel structures are maintained in `st.session_state`. `chat_history` is a flat list of role/content dicts, passed to the LLM for context (last 6 turns for answers, last 4 for classification). `chat_messages` is a richer list including article objects, source lists, and response type flags — used exclusively for UI rendering. This separation avoids sending large article objects to the LLM unnecessarily.

5.4 Session Continuity

The New Chat button navigates to the welcome screen (sets `in_chat=False`) without clearing `chat_history` or `chat_messages`. The sidebar continues to show the session's recent queries. History is only lost on browser tab close or refresh, which resets all Streamlit session state. This mirrors commercial chatbot behaviour and was a deliberate design decision to provide a better user experience.

6. API and Web Search Integration

6.1 NewsAPI Integration

Endpoint	Used For	Key Parameters
/v2/top-headlines	Topic pills, Today's Top News, bootstrap	country=us, category, language=en, pageSize=10
/v2/everything	Free-text topic keyword search	q=<topics>, language=en, sortBy=publishedAt, pageSize=10

Table 6.1 — NewsAPI Endpoints

Both endpoints always request `pageSize=10`. This is intentional — the credibility filter needs a buffer of articles to drop UNRELIABLE ones while still returning up to five results to the user. If only five were fetched and two were dropped, only three would be shown. All API calls read `NEWS_API_KEY` live from `os.getenv()` on every call so the key is always current after `load_dotenv()` runs. Articles with "[Removed]" in the title are filtered out during normalisation.

6.2 Landing Page Direct Calls

Three landing page helper functions call NewsAPI directly, bypassing the LangGraph graph. This is intentional: the welcome screen article preview does not require query classification or LLM involvement. Crucially, all three helpers call `score_and_filter()` after fetching — ensuring landing page articles pass the same credibility bar as chat-path articles. The user sees consistently filtered content regardless of which surface they are viewing.

6.3 Tavily Web Search

Tavily serves two roles. As a news fallback in `fetch_news`, it is called when NewsAPI returns no results or fails, with results normalised into the same NewsArticle shape. As a general query enricher in `llm_answer`, it is called with `search_depth='advanced'` before the LLM answer call, providing up to five web snippets as RAG context. This grounds the LLM's answer in current web content rather than relying solely on training data.

6.4 Credential Management

All API keys are stored in a `.env` file and loaded by `python-dotenv`'s `load_dotenv()` at the top of both `main.py` and `agents.py`. Keys are read via `os.getenv()` at the point of use — not stored in module-level variables that could be evaluated before `load_dotenv()` runs. The optional `OPENAI_BASE_URL` variable supports proxy environments such as Vocareum.

7. LLM-Powered Credibility Filtering

7.1 Design Decision — Why LLM Over a Separate ML Model

An earlier version of this system used dhruvpal/fake-news-bert, a DistilBERT model fine-tuned for fake news detection from Hugging Face. After review, this approach was replaced with an LLM-based scoring system for the following reasons:

- The DistilBERT model required torch, transformers, and torchvision as dependencies — adding approximately 250MB of downloads and 30-second first-run model loading. The LLM approach adds zero new dependencies since OpenAI is already required.
- The HuggingFace model was trained on a specific dataset from 2017-2018 and produced black-box scores with no explanation. The LLM approach produces a one-sentence reason per article that is displayed to the user — making the filtering transparent and explainable.
- The DistilBERT model detected stylistic patterns associated with fake news writing. GPT-4o-mini reasons about credibility signals — source attribution, claim specificity, language tone, headline integrity — which is closer to what a human editor does and more appropriate for modern news writing styles.
- The LLM approach is fail-open: if scoring fails for any reason, all articles are returned unscored and the user still sees results. The HuggingFace approach had additional failure modes around model loading and GPU/CPU compatibility.

7.2 Credibility Assessment Signals

The `score_and_filter()` function sends all fetched articles to GPT-4o-mini in a single batch call. The system prompt instructs the model to assess each article against four signals:

Signal	What is assessed
Language tone	Neutral and factual vs sensationalist, emotionally manipulative, or alarmist language
Source attribution	Is the claim attributed to a named, verifiable person or organisation, or anonymous?
Claim specificity	Specific facts — dates, numbers, named entities — vs vague or unverifiable assertions
Headline integrity	Does the headline accurately represent the description? Are clickbait patterns present?

Table 7.1 — Credibility Assessment Signals

7.3 Scoring and Filtering Logic

The batch prompt sends all articles as a numbered list and instructs the model to return a JSON array — one object per article — with three fields: index, label, and reason. Using a single batch call rather than one call per article keeps latency and cost proportional regardless of batch size.

Labels and their handling:

Label	Meaning	Action	UI Display
RELIABLE	Passes all four signals	Kept · ranked first	● Reliable + reason
UNCERTAIN	Mixed signals, insufficient data	Kept · ranked second	○ Uncertain + reason
UNRELIABLE	Fails credibility assessment	Dropped before display	Never shown to user

Table 7.2 — Label Handling

7.4 Why Fetch 10, Show Up to 5

The system always fetches 10 articles from NewsAPI (pageSize=10) even though only 5 are shown. This buffer is essential for the filter to work correctly. If only 5 were fetched and 3 were UNRELIABLE, only 2 would be shown — a poor user experience. Fetching 10 means that even with 5 articles dropped, the user still sees 5 credible results. If fewer than 5 survive after filtering, the user is notified with a message such as '3 articles filtered for credibility' in the response summary line.

7.5 Consistency Across All Surfaces

Credibility filtering runs in two places. The score_news LangGraph node filters articles on the chat news path. The three landing page helper functions (fetch_landing_top_headlines, fetch_landing_for_topic, fetch_landing_for_quick) each call score_and_filter() directly after fetching, before returning articles to the UI. This ensures the user sees identically filtered content whether they are browsing the welcome screen or asking a question in chat.

7.6 Important Limitations

- Input truncation: only title and description are scored. Full article body text, which would provide stronger credibility signals, is not available from the NewsAPI free tier.
- Not a fact-checker: the LLM assesses writing style and attribution signals, not factual accuracy. A well-written fabrication may score RELIABLE. A poorly formatted legitimate article may score UNCERTAIN.
- LLM bias: the scoring reflects GPT-4o-mini's training data and may carry biases toward certain writing styles, topics, or source conventions.
- Fail-open by design: if the scoring call fails (network error, quota exhaustion), all articles are returned unscored with credibility_label=None. The user sees the articles without badges rather than seeing an error or empty results.

8. Session Memory and State Management

8.1 Streamlit Session State

NewsGenie implements session memory entirely through `st.session_state`, which persists across reruns within a single browser session. No external database is used. The following keys are maintained:

Key	Type	Contents and Lifecycle
<code>chat_history</code>	List[dict]	Flat {role, content} list. Passed to LLM for context. Never cleared by New Chat.
<code>chat_messages</code>	List[dict]	Rich message objects with articles, sources, flags. Used for UI rendering only.
<code>in_chat</code>	bool	Routes between welcome screen (False) and chat view (True). Reset by New Chat.
<code>graph</code>	CompiledGraph	Compiled LangGraph graph. Built once at startup, cached for session lifetime.
<code>landing</code>	dict	Landing page state: mode, quick_id, topic_key, articles, error, bootstrapped.
<code>_last_latest</code>	str	Tracks selectbox value to prevent re-triggering on unrelated reruns.

Table 8.1 — Session State Keys

8.2 LangGraph State vs Session State

The LangGraph `NewsGenieState` is a per-query object created fresh by `empty_state()` for every `run_query()` call. Chat history from session state is injected as a list copy — `list(st.session_state['chat_history'][-12:])` — before `graph.invoke()` is called. This copy survives LangGraph's internal state reconstruction between nodes. After the graph returns, results are appended to the session state lists and `st.rerun()` produces a clean render. The two state systems never overlap.

9. Error Handling and Fallback Chain

9.1 Three-Level Fallback

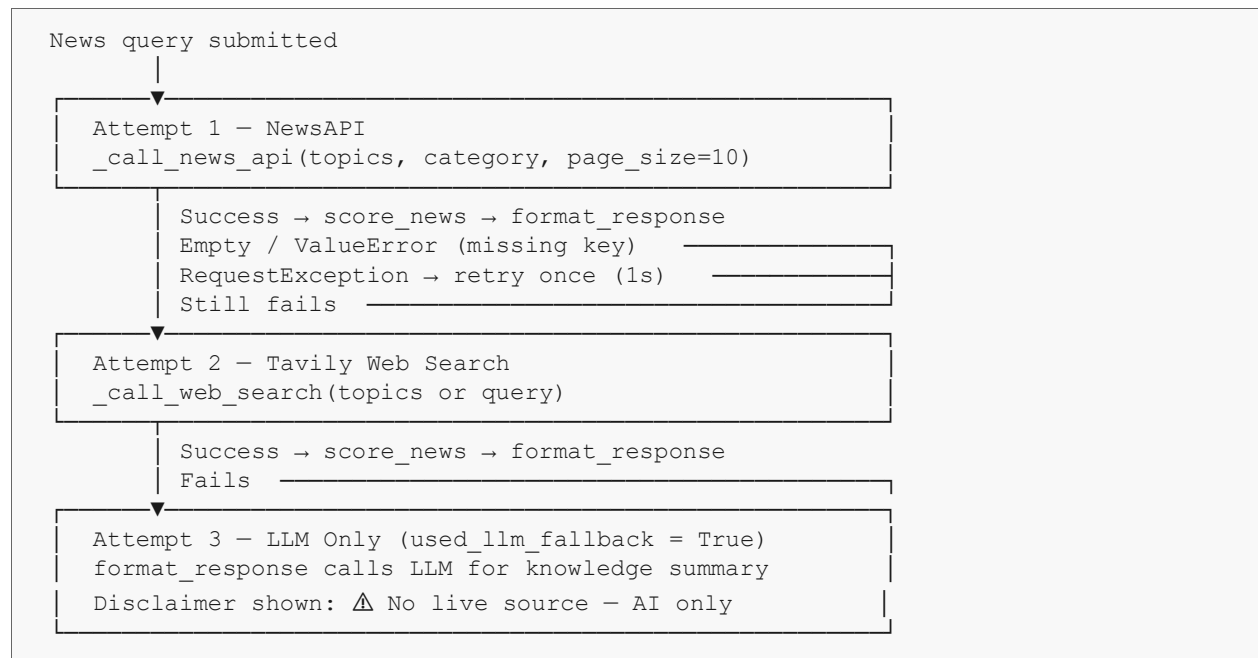


Figure 9.1 — Three-Level Fallback Chain

9.2 Error Scenarios

Error	Detection	Response
Missing NEWS_API_KEY	ValueError on empty os.getenv()	Warning set; immediately falls back to Tavily
NewsAPI timeout / 5xx	requests.RequestException	Retry once after 1 second, then fall back to Tavily
Empty API results	len(articles) == 0	Skip to Tavily without retry
Missing TAVILY_API_KEY	ValueError on empty os.getenv()	Tavily skipped; fall through to LLM fallback
All sources fail	used_llm_fallback = True	LLM knowledge summary with disclaimer badge
LLM quota exhausted	_is_budget_error() check	Specific budget error message shown to user
Credibility scoring fails	Exception in score_and_filter()	Fail-open: all articles returned unscored, no crash
Broken image URL	try/except around st.image()	Image silently skipped; text content shown normally

Table 9.1 — Error Scenarios

10. User Interface Design

10.1 Welcome Screen

- Brand header — 'NewsGenie' with tagline 'Your trusted source for credibility-filtered news.'
- Time-aware greeting — Good morning / afternoon / evening based on server hour.
- Quick actions — 'Today's Top News' and 'What's Trending' buttons. Each shows a contextual spinner while fetching and scoring articles.
- Latest update on... — selectbox for predefined topics (AI, US Politics, Global Markets, etc.) that fires `run_query()` and enters chat mode.
- Topic pills — Business, Technology, Sports, Health, Entertainment. Each pill fetches 10 articles, scores them, and displays up to 5 filtered results with a spinner. Active pill is highlighted.
- Article preview list — up to 5 articles as collapsed expanders. Each shows credibility badge (●/○), reason, source, publication date, description, and 'Read full article' link.
- Chat input — `st.chat_input()` pinned to bottom with placeholder 'What do you want to know?'

10.2 Chat View

- User messages rendered as `st.chat_message('user')`.
- Assistant messages rendered with  avatar, containing: error/warning banners, fallback disclaimer badge if applicable, response summary text, article expander cards with credibility badges and reasons, and source chips.
- Follow-up chat input with placeholder 'Ask a follow-up question...'

10.3 Sidebar

- NewsGenie logo and New Chat button — navigates to welcome screen, preserves chat history.
- Recent chats — last six user query strings shown for session reference.

10.4 Double-Render Bug Fix

During testing, topic pill and quick action button clicks caused the button row to render twice. Root cause: `_apply_landing_fetch()` was modifying `st.session_state` mid-render, causing Streamlit to schedule a surprise rerun before the first render completed. Fix: `st.rerun()` added as the last line of `_apply_landing_fetch()`, and the bootstrap block restructured to set `bootstrapped=True` before the spinner runs. This ensures state mutations always happen before or after render, never during.

11. Execution Flow — End-to-End Trace

Scenario: User types 'What is the latest news about AI regulation?'

7. Browser loads the app. `main.py` executes. `load_dotenv()` loads API keys. `init_session()` initialises all session state keys. `build_graph()` compiles the LangGraph StateGraph and caches it.
8. `render_welcome()` runs. Landing bootstrap fires: `fetch_landing_top_headlines()` fetches 10 articles, `score_and_filter()` scores and filters them, up to 5 are displayed in the welcome screen.
9. User types query. `run_query('What is the latest news about AI regulation?')` is called. `empty_state()` creates a fresh NewsGenieState. `list(st.session_state['chat_history'][-12:])` is injected. `graph.invoke(state)` is called.
10. Node 1 — `classify_query`: `selected_category` is None. GPT-4o-mini returns `{"type": "news", "topics": ["AI regulation"]}`. State updated: `query_type='news'`, `extracted_topics=['AI regulation']`.
11. `route_after_classify()` returns 'fetch_news'. Graph routes to `fetch_news`.
12. Node 2 — `fetch_news`: `_call_news_api(['AI regulation'], None, page_size=10)` fetches 10 articles from `/v2/everything`. Articles normalised. `news_articles` set with up to 10 items.
13. Node 4 — `score_news`: `score_and_filter(articles)` sends all 10 to GPT-4o-mini in one batch call. Model returns RELIABLE/UNCERTAIN/UNRELIABLE labels with reasons. UNRELIABLE articles dropped. Survivors ranked RELIABLE first. `filtered_count` recorded.
14. Node 5 — `format_response`: `articles[:5]` selected. Sources deduplicated. Summary string built including filter count note. `response_text`, `response_articles`, `response_sources`, `response_type` returned.
15. `graph.invoke()` returns. `run_query()` appends to `chat_history` and `chat_messages`. `in_chat = True`. `st.rerun()` fires.
16. `render_chat()` renders the conversation. Each article card shows credibility badge and reason. Source chips shown below article list.

12. Sample Outputs

12.1 News Query — Topic Pill Click

User action: Clicked 'Technology' topic pill
 Trigger: `fetch_landing_for_topic('technology')` called directly
 Fetch: 10 articles from `/v2/top-headlines?category=technology`
 Scoring: `score_and_filter()` — 1 article dropped (UNRELIABLE: sensationalist headline)
 Displayed: 5 articles, ranked RELIABLE first
 Badges: #1  Reliable #2  Reliable #3  Uncertain #4  Reliable #5  Uncertain

12.2 News Query — Free Text

User input: 'What happened at the G7 summit this week?'
 Classification: `{"type": "news", "topics": ["G7 summit"]}`
 Fetch: 10 articles from `/v2/everything?q=G7+summit`
 Scoring: 2 articles dropped. Summary: 'Here are the most credible stories on G7 summit. 2 articles filtered for credibility.'
 Displayed: 5 articles, RELIABLE ranked before UNCERTAIN

12.3 General Query

User input: 'What is LangGraph and how does it work?'
 Classification: `{"type": "general", "topics": []}`
 Path: `llm_answer` node — no NewsAPI call
 Tavily: 5 web snippets retrieved (`search_depth='advanced'`)
 Response: GPT-4o-mini synthesises answer from snippets + knowledge
 Sources: `python.langchain.com · github.com` (shown as source chips)

12.4 Fallback Scenario

Trigger: NEWS_API_KEY missing from `.env`
 Attempt 1: `ValueError: 'NEWS_API_KEY not set'` — skip to Tavily
 Attempt 2: TAVILY_API_KEY also missing — skip to LLM
 Response: LLM generates knowledge summary
 UI: `st.error('Could not retrieve live news...')`
 ⚠ disclaimer badge: 'No live source — AI knowledge only'

13. Limitations and Future Enhancements

13.1 Current Limitations

- NewsAPI free tier: 100 requests per day, articles from last 30 days only. Production deployment requires a paid tier.
- Title and description only: credibility scoring uses the article title and description. Full body text would provide significantly stronger signals but is not available on the free tier.
- LLM credibility scoring: the LLM assesses writing style and attribution, not factual accuracy. A well-written fabrication may still score RELIABLE.
- In-session memory only: chat history and interest data are lost on browser close or refresh. No cross-session persistence.
- English-language only: both the LLM prompts and NewsAPI parameters specify English.

13.2 Future Enhancements

- Persistent storage: store chat history in SQLite or Redis for cross-session continuity.
- Full article body scoring: integrate a paid news API or web scraper to retrieve full article text for more accurate credibility assessment.
- Source reputation layer: cross-reference article sources against established media credibility databases (e.g. Media Bias/Fact Check) as a second credibility signal.
- Personalized suggestions (Phase 2): reintroduce session-based interest scoring to drive the suggested story cards on the welcome screen.
- Multi-language support: extend to Spanish, French, and other major languages using multilingual prompts and NewsAPI language parameters.

14. Conclusion

NewsGenie delivers a unified conversational platform that addresses all three components of the project brief: intelligent query handling through LLM-based classification, real-time data through NewsAPI and Tavily, and credibility filtering through LLM-based article scoring.

The credibility filtering system represents the most significant architectural decision of the project. Using an LLM-based approach rather than existing HuggingFace DistilBERT classifier eliminated 250MB of dependencies, removed GPU/CPU compatibility concerns, and produced an explainable, transparent result — a one-sentence reason per article that the user can read and evaluate. Most importantly, it moved from badging misinformation to actually filtering it: UNRELIABLE articles are dropped before the user sees them, directly satisfying the brief's requirement to 'filter out unreliable or misleading content.'

The modular four-file architecture — `state.py`, `agents.py`, `workflow.py`, `main.py` — keeps responsibilities cleanly separated. The LangGraph StateGraph makes the query routing explicit and auditable. The three-level fallback chain ensures the user always receives a response even when external services fail.

The project demonstrates practical competency in: LangGraph stateful graph orchestration, LLM prompt engineering for classification and structured output, RAG-style answer generation, multi-level API fallback design, Streamlit session management, and the design of transparent, explainable AI filtering systems.

Appendix A — Dependencies and Setup

A.1 requirements.txt

```
streamlit>=1.32.0
langgraph>=0.1.0
langchain>=0.2.0
langchain-openai>=0.1.0
langchain-core>=0.2.0
openai>=1.30.0
requests>=2.31.0
tavily-python>=0.3.0
python-dotenv>=1.0.0
```

A.2 API Keys Required

Variable	Source	Notes
OPENAI_API_KEY	platform.openai.com	Required. GPT-4o-mini for all LLM calls.
NEWS_API_KEY	newsapi.org/register	Required. Free tier: 100 req/day.
TAVILY_API_KEY	app.tavily.com	Required. Free tier sufficient for development.
OPENAI_BASE_URL	Your proxy URL	Optional. For Vocareum or proxy environments.

A.3 Run Instructions

- Copy env.example to .env and add your API keys.
- Install dependencies: `pip install -r requirements.txt`
- Run: `streamlit run main.py`
- Browser opens at `http://localhost:8501`

A.4 File Structure

```
newsgenie/
├── main.py           # Streamlit frontend
├── workflow.py       # LangGraph StateGraph
├── agents.py         # Node functions, scorer, landing helpers
├── state.py          # TypedDicts and empty_state()
├── requirements.txt  # Python dependencies
└── .env              # API keys (never commit to Git)
```